RAG-BASED CUSTOMER SUPPORT ASSISTANT

Technical Documentation

E-Commerce Support Intelligence System

Internship Project | Version 1.0

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines two capabilities: information retrieval and language generation. A standard LLM answers questions purely from patterns learned during training — it cannot access your specific documents, policies, or databases. RAG solves this by splitting the problem into two steps.

First, a retrieval step: when a query arrives, the system searches a pre-indexed document store and fetches the most relevant passages. Second, a generation step: those passages are injected into the LLM's context window along with the query, and the model generates an answer grounded in that specific content.

The key innovation is that the LLM never needs to memorise your documents. It only needs to read and reason over the retrieved passages at inference time. This makes RAG systems easy to update — changing the knowledge base requires only re-indexing the documents, not retraining the model.

1.2 Why RAG is Needed Here

- An e-commerce FAQ PDF contains policies, procedures, and product information that a generic LLM does not know
- The LLM cannot reliably answer 'What is our return window?' without access to the actual policy document
- Fine-tuning an LLM on the FAQ would be expensive and would require retraining whenever policies change
- RAG allows the system to answer accurately from the PDF without any model training
- When the FAQ is updated, only re-running ingest.py is needed — no code changes

1.3 Use Case Overview

This system serves as an intelligent first-line support agent for an e-commerce platform. Customers can ask natural language questions about their orders, returns, shipping, payments, and accounts. The system classifies each query, retrieves relevant FAQ content, and generates a clear answer. Queries that are vague, out-of-scope, or too complex are handled gracefully through clarification requests, scope rejections, or human escalation — all automatically, without any manual routing rules.

2. System Architecture Explanation

2.1 Two-Pipeline Architecture

The system operates in two separate pipelines that run at different times and serve different purposes.

The Ingestion Pipeline runs once, offline, before the application starts. It reads the PDF, splits it into chunks, converts each chunk into an embedding vector, and stores everything in ChromaDB on disk. This pipeline is triggered by running python ingest.py and does not need to run again unless the PDF changes.

The Query Pipeline runs in real time for every user message. It takes the user's raw text, routes it through a LangGraph state machine, and produces a final answer. The graph has up to 5 nodes executing per query depending on the route taken, with each node performing a well-defined operation.

2.2 Component Interactions

Component A	Interacts With → How
app.py (Streamlit)	graph.py → calls run_rag(query), receives result dict, updates session state
graph.py (LangGraph)	retriever.py → calls get_retriever() once at startup to get ChromaDB retriever
graph.py intent_node	Groq API → sends classification prompt, receives JSON intent
graph.py retrieve_node	ChromaDB → retriever.invoke(query) returns top-4 Document objects
graph.py relevance_check_node	Groq API → sends chunks + query, receives 'relevant'/'irrelevant'
graph.py generate_node	Groq API → sends context + query with system prompt, receives answer
retriever.py	ChromaDB (chroma_db/) → loads persisted vector store from disk
ingest.py	PDF file → PyPDFLoader reads it; SentenceTransformers embeds it; ChromaDB stores it

3. Design Decisions

3.1 Chunk Size: 500 characters, 100 overlap

Chunk size directly affects retrieval quality and answer completeness. A chunk that is too small (e.g., 100 chars) may not contain a complete thought — an answer about a return policy might require three consecutive sentences to be complete, and splitting them into separate chunks means the retriever may only fetch one. A chunk that is too large (e.g., 2000 chars) dilutes the semantic signal, causing the embedding to represent a mixture of topics rather than a specific concept.

500 characters was chosen as a balance point for FAQ-style documents. FAQ entries are typically 2-5 sentences long — 500 characters captures one complete FAQ answer in most cases. The 100-character overlap ensures that answers split at a chunk boundary appear in both adjacent chunks, so the retriever can surface the complete answer regardless of where the split occurred.

3.2 Embedding Strategy: Local SentenceTransformers

The all-MiniLM-L6-v2 model was chosen for three reasons: it runs entirely locally (no API cost per embedding call), it was specifically trained for semantic similarity tasks (not just general language modelling), and it produces compact 384-dimensional vectors that are fast to index and search. Alternatives considered were OpenAI's text-embedding-ada-002 (better quality but $0.0001 per 1K tokens, adds API dependency) and simple TF-IDF (fast but keyword-only, misses synonyms and paraphrases).

3.3 Retrieval Approach: Dense Vector Similarity

ChromaDB retrieves the top-4 chunks by cosine similarity between the query embedding and all stored chunk embeddings. The choice of k=4 means the LLM receives approximately 2000 characters of context (4 chunks x ~500 chars). This is well within Llama 3.3's 128K context window, but provides enough coverage to answer multi-part questions. A relevance check node then validates these chunks before generation, adding a quality gate that reduces hallucination.

3.4 Prompt Design Logic

Generate Node System Prompt (actual prompt used)

You are a professional e-commerce customer support assistant.
Rules: - Answer using ONLY the provided context. Do not make up information.
- Be friendly, concise, and actionable.
- If the context partially answers the question, give what you can and clearly `state what you don't have information about.` - If the context does not answer the question at all, respond with exactly one
`line: 'INSUFFICIENT_CONTEXT'`
- Never guess order details, prices, or policies not mentioned in context.

The INSUFFICIENT_CONTEXT sentinel is critical. It gives the system a machine-readable signal to trigger escalation — a natural language 'I don't know' response would require fuzzy string matching, which can produce false positives. The strict single-line format makes the confidence check deterministic.

4. Workflow Explanation

4.1 Why LangGraph Instead of a Simple Chain

A simple LangChain chain executes steps linearly: A → B → C → D. This works for fixed pipelines but breaks down when you need conditional branching. Our system has at least 5 different paths a query can take depending on its intent, chunk relevance, and LLM confidence. Implementing this as nested if/else blocks in plain Python would produce tangled, hard-to-maintain code.

LangGraph solves this by making the control flow explicit. Each path is a named edge between named nodes. The entire graph can be visualised, tested node-by-node, and extended without touching existing nodes. Adding a new routing case (e.g., a 'policy_change' route) requires only adding one node and one conditional edge.

4.2 Node Responsibilities Summary

Node	Single Responsibility
intent_node	Classify the query into one of four intent categories using the LLM
out_of_scope_node	Return a polite out-of-scope rejection message
clarify_node	Generate a smart clarifying question for vague inputs
critical_escalation_node	Flag critical queries and return a high-priority escalation message
retrieve_node	Fetch the top-4 relevant chunks from ChromaDB
relevance_check_node	Validate that retrieved chunks are actually relevant to the query
generate_node	Generate an answer from the context and detect low-confidence responses
hitl_node	Replace the answer with an escalation message when confidence is low
output_node	Mark the route as 'answer' and pass state to the caller

4.3 State Transitions

The GraphState TypedDict object is initialised with empty/default values before the graph runs. Each node reads from and writes to this shared state object. No node receives arguments directly — all data comes from state, and all output is written back to state. This makes every

node independently testable: you can construct a state dict manually and call any node function directly without running the full graph.

5. Conditional Logic

5.1 Intent Detection — LLM vs Rules

The previous version of this system used regex patterns for intent detection. While fast, regex is brittle: 'charged twice' would be caught, but 'they billed me double' would not. The upgraded system uses the LLM itself as the classifier, which understands paraphrase, context, and intent regardless of exact wording.

The classifier uses temperature=0 (fully deterministic) and is instructed to return only JSON. This means for any given input string, the output intent will be the same every time — critical for a production system where unpredictable behaviour is unacceptable. The system prompt includes concrete examples for each category to guide the model.

5.2 Routing Decisions

Decision tree for every incoming query
Query arrives └ intent_node classifies query
├ 'out_of_scope' → out_of_scope_node → END ├ 'ambiguous' → clarify_node → END
├ 'critical' → critical_escalation_node → END └ 'in_scope' → retrieve_node └ relevance_check_node ├ irrelevant → hitl_node → END └ relevant → generate_node ├ low confidence → hitl_node → END
└ high confidence → output_node → END

6. HITL Implementation

6.1 Role of Human Intervention

The Human-in-the-Loop design acknowledges a fundamental truth about AI systems: they will encounter queries they cannot reliably answer. Rather than returning a wrong or fabricated answer, this system identifies those boundaries and defers to a human. This is especially important in e-commerce, where an incorrect answer about a refund policy or billing dispute can cause real financial harm or customer churn.

HITL in this system serves three distinct functions: handling genuinely critical issues that require human judgment (fraud, billing disputes), handling knowledge gaps where the AI cannot find an answer in the knowledge base, and creating a feedback loop opportunity where escalated queries can be reviewed to identify gaps in the FAQ document.

6.2 Benefits of the Current HITL Design

- Zero hallucination on critical issues — critical queries never reach the LLM generation step
- Transparent to the user — the escalation message clearly explains what happened and sets expectations
- Auditable — needs_human=True and route='escalate' are stored in session state, enabling analysis
- Graceful degradation — a system failure (LLM error, empty chunks) also routes to HITL rather than crashing

6.3 Limitations and Future Improvements

- **Limitation:** Current HITL is simulated — no real ticket is dispatched to a human agent CRM
- **Limitation:** No session continuity — if a user returns after human intervention, the chat history is lost
- **Limitation:** No learning loop — escalated queries are not fed back to improve the knowledge base
- **Future:** Integrate Zendesk/Freshdesk API in critical_escalation_node and hitl_node to create real tickets
- **Future:** Add a feedback button after each AI answer so users can flag incorrect responses

- **Future:** Log all escalated queries to a database and review weekly to identify FAQ gaps

7. Challenges and Trade-offs

Trade-off	Analysis & Decision
Retrieval Accuracy vs Speed	Larger embedding models (e.g., text-embedding-3-large) produce more accurate retrieval but add API cost and latency per query. all-MiniLM-L6-v2 runs locally in ~50ms and produces good results for FAQ-style English text. For a production system handling multiple languages or technical jargon, upgrading the embedding model would be the first improvement.
Chunk Size vs Context Quality	Larger chunks (800-1000 chars) reduce the risk of splitting an answer across chunks, but the embedding represents a broader topic mix, reducing retrieval precision. Smaller chunks (200-300 chars) give precise retrieval but may miss context. 500 characters with 100 overlap was chosen as the sweet spot for single-topic FAQ entries.
LLM Calls per Query (Cost vs Quality)	The current system makes up to 3 LLM calls per in-scope query: classification, relevance check, and generation. At Groq's free tier speed, this adds ~1.5 seconds of latency. The relevance check could be removed to save one call, but at the cost of occasional hallucinations when irrelevant chunks are passed to the generator. The extra call was kept because answer quality is more important than 500ms of speed.
Local Embeddings vs Cloud Embeddings	Running SentenceTransformers locally eliminates per-call embedding costs and removes the API dependency. The trade-off is that the model must be downloaded (~90MB) on first run and consumes local CPU during embedding. For this project, the cost savings and offline capability outweigh the setup friction.
LangGraph Complexity vs Flexibility	LangGraph adds architectural complexity compared to a simple Python function. However, as the routing logic grew from 2 paths to 5+ paths, the graph structure became essential for maintainability. Without LangGraph, the routing logic would be a deeply nested if/else block in a single function.

8. Testing Strategy

8.1 Functional Test Cases

Query	Expected Route & Behaviour
Where is my order?	in_scope → RAG pipeline → answer (green pill). Should retrieve shipping/tracking FAQ chunks.
How do I return an item?	in_scope → RAG pipeline → answer. Should retrieve return policy chunks.
payment	ambiguous → clarify_node. Should ask what specifically the user needs help with regarding payment.
help	ambiguous → clarify_node. Very vague — should ask for more detail.
I was charged twice	critical → critical_escalation_node. Must NEVER reach RAG pipeline.
My account was hacked	critical → critical_escalation_node. High-severity issue, immediate escalation.
I want a refund	critical → critical_escalation_node. Refund = financial dispute = human required.
What is the weather today?	out_of_scope → rejection. Completely unrelated to e-commerce.
Write me a poem about shipping	out_of_scope → rejection. Creative request, not support query.
What is the capital of France?	out_of_scope → rejection. General knowledge, not support.
Do you offer free shipping on orders over 500?	in_scope → RAG. If answer is in FAQ, returns it. If not in FAQ, escalates.
Can I cancel my order after it has been shipped?	in_scope → RAG. Tests complex policy question retrieval.

8.2 Edge Case Testing

- Empty input string — should not crash, intent classifier should handle it gracefully
- Query in a non-English language — classifier should recognise it as potentially in-scope and attempt RAG
- Extremely long query (500+ words) — should not exceed LLM token limits, may need truncation in future
- SQL injection or prompt injection attempt — LLM system prompt is sandboxed, should not execute injected instructions

- Query about a topic covered in the FAQ but using unusual synonyms — tests embedding model's semantic coverage

9. Future Enhancements

Enhancement	Description & Implementation Path
Multi-Document Support	Extend ingest.py to accept a directory of PDFs. Add a document_id metadata field to each chunk so that different knowledge bases (e.g., returns policy, product catalogue, shipping info) can be indexed separately and queried together or independently.
Feedback Loop	Add a thumbs up/down button after each AI answer in the Streamlit UI. Log negative feedback with the query, retrieved chunks, and answer to a SQLite database. Review weekly to identify knowledge gaps. Use flagged queries as a test set for regression testing after FAQ updates.
Conversation Memory	Integrate LangChain's ConversationBufferWindowMemory or a custom memory node in LangGraph to maintain context across multiple turns. This would allow follow-up questions like 'And what about international returns?' to reference the previous exchange.
Real HITL Integration	Replace the escalation placeholder with a Zendesk or Freshdesk API call. Create a ticket on escalation, store the ticket ID in session state, and poll for the human response. Inject the agent's reply into the chat when it arrives.
Streaming Responses	Use Groq's streaming API to display the answer token-by-token instead of waiting for the full response. This reduces perceived latency significantly. LangChain's ChatGroq wrapper supports streaming via callbacks.
Evaluation Framework	Build an automated eval harness using RAGAS (Retrieval Augmented Generation Assessment). Measure faithfulness (answer grounded in context?), answer relevancy, and context recall. Run evals automatically when the FAQ is updated.
Production Deployment	Containerise the application with Docker. Deploy ChromaDB as a separate service (or migrate to Pinecone/Qdrant for horizontal scaling). Deploy the Streamlit app to a cloud platform (AWS ECS, GCP Cloud Run, or Hugging Face Spaces). Add authentication via Streamlit's built-in auth or an OAuth provider.
Analytics Dashboard	Add a second Streamlit page with charts showing query volume over time, route distribution (how many escalations vs answers), most common topics retrieved, and average response latency. Store all query data in SQLite or PostgreSQL.